Digital Wallet and UPI Payment System

Java Assignment – Object-Oriented Programming

1. Objective / Aim

To design and implement a Digital Wallet and UPI Payment System in Java using interfaces, classes, encapsulation, custom exceptions, and exception handling, simulating real-world digital payment operations such as wallet creation, adding money, UPI payments, balance checking, and transaction validation.

2. Problem Description

Design a digital wallet system for a user. The system should allow the user to create a wallet with a name, mobile number, and UPI ID; add money to the wallet; make payments using another user's UPI ID; check the available balance; validate the receiver's UPI ID; prevent payments when the balance is insufficient; reject zero or negative amounts; handle invalid transactions using appropriate exceptions; and display the final wallet details and transaction status.

3. Concepts Demonstrated

Concept	Implementation
Interface	PaymentService
Encapsulation	Private data members in Wallet
Constructor	Initializes wallet details
Class implementation	UPIPayment implements PaymentService
Custom exceptions	Three user-defined exception classes
Exception handling	try-catch-finally
String operations	matches() validates the UPI ID
Methods	addMoney(), pay(), checkBalance(), displayWalletDetails()

4. Algorithm / Logic Description

1. Start the program.
2. Create the PaymentService interface and declare pay() and checkBalance().
3. Create the Wallet class with private data members: user name, mobile number, UPI ID, and balance.
4. Define a constructor to initialize the wallet details.
5. Define addMoney(), getBalance(), deductMoney(), and displayWalletDetails().
6. Create the UPIPayment class and implement the PaymentService interface.
7. Create the custom exceptions InsufficientBalanceException, InvalidUPIException, and InvalidAmountException.
8. Create a Wallet object and associate it with an UPIPayment object.
9. Read the user's wallet details and the amount to add.
10. Reject the amount if it is zero or negative; otherwise add it to the wallet.
11. Read the receiver's UPI ID and payment amount.
12. Validate the receiver's UPI ID using a String regular expression.
13. If the UPI ID is invalid, throw InvalidUPIException.
14. If the payment amount is zero or negative, throw InvalidAmountException.

15. If the payment amount is greater than the available balance, throw InsufficientBalanceException.
16. If all validations are successful, deduct the payment amount and display a success message.
17. Use catch blocks to display the appropriate transaction failure message.
18. Use finally to display the final wallet details and balance.
19. Stop the program.

5. UML Class Diagram

```
Savings/other inheritance not used in this program.

Custom Exceptions:
InsufficientBalanceException extends Exception
InvalidUPIException extends Exception
InvalidAmountException extends Exception
```

Relationship: UPIPayment implements PaymentService and has an association with Wallet. The three custom exception classes inherit from Java's Exception class.

6. Class and Interface Design

Component	Purpose
PaymentService	Declares pay() and checkBalance().
Wallet	Stores user information and wallet balance using private fields.
UPIPayment	Performs UPI payment and balance checking.
InsufficientBalanceException	Raised when the wallet balance is insufficient.
InvalidUPIException	Raised when the receiver UPI ID is invalid.
InvalidAmountException	Raised when an amount is zero or negative.

7. Source Code with Comments

```
import java.util.Scanner;

// Interface
interface PaymentService {
    void pay(String receiverUPI, double amount)
            throws InvalidUPIException,
                   InvalidAmountException,
                   InsufficientBalanceException;

    void checkBalance();
}

// Custom Exception 1
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

// Custom Exception 2
class InvalidUPIException extends Exception {
    InvalidUPIException(String message) {
        super(message);
    }
}

// Custom Exception 3
class InvalidAmountException extends Exception {
    InvalidAmountException(String message) {
        super(message);
    }
}

// Wallet Class
class Wallet {
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    // Constructor
    Wallet(String userName, String mobileNumber,
           String upiId, double balance) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = balance;
    }

    // Add money to the wallet
    void addMoney(double amount) {
        balance += amount;
        System.out.println("Money added successfully: Rs. " + amount);
    }

    // Return current balance
    double getBalance() {
        return balance;
    }

    // Deduct money after successful validation
    void deductMoney(double amount) {
        balance -= amount;
    }

    // Display wallet details
    void displayWalletDetails() {
        System.out.println("\n----- WALLET DETAILS -----");
        System.out.println("User Name: " + userName);
        System.out.println("Mobile Number: " + mobileNumber);
        System.out.println("UPI ID: " + upiId);
        System.out.println("Balance: Rs. " + balance);
    }
}

// UPI Payment Class
class UPIPayment implements PaymentService {
    private Wallet wallet;

    // Constructor associates a wallet with the payment service
    UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }
```

```
    // Make a UPI payment
    @Override
    public void pay(String receiverUPI, double amount)
            throws InvalidUPIException,
                   InvalidAmountException,
                   InsufficientBalanceException {

        // Validate receiver UPI ID
        if (receiverUPI == null
                || !receiverUPI.matches(
                "[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+")) {
            throw new InvalidUPIException(
                    "Invalid UPI ID! Payment cancelled.");
        }

        // Validate payment amount
        if (amount <= 0) {
            throw new InvalidAmountException(
                    "Invalid amount! Amount must be positive.");
        }

        // Check available balance
        if (amount > wallet.getBalance()) {
            throw new InsufficientBalanceException(
                    "Insufficient balance! Payment cancelled.");
        }

        // Deduct money after all validations succeed
        wallet.deductMoney(amount);

        System.out.println("Payment successful!");
        System.out.println("Receiver UPI: " + receiverUPI);
        System.out.println("Amount paid: Rs. " + amount);
    }

    // Check wallet balance
    @Override
    public void checkBalance() {
        System.out.println("Available Balance: Rs. "
                + wallet.getBalance());
    }
}

// Main Class
public class DigitalWallet {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("===== DIGITAL WALLET SYSTEM =====");

        // Enter your own details
        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter mobile number: ");
        String mobile = sc.nextLine();

        System.out.print("Enter your UPI ID: ");
        String upi = sc.nextLine();

        // Create wallet and payment objects
        Wallet wallet = new Wallet(name, mobile, upi, 0);
        UPIPayment payment = new UPIPayment(wallet);

        try {
            System.out.print("Enter amount to add: ");
            double addAmount = sc.nextDouble();

            if (addAmount <= 0) {
                throw new InvalidAmountException(
                        "Invalid amount! Cannot add money.");
            }

            wallet.addMoney(addAmount);
            payment.checkBalance();

            sc.nextLine();

            System.out.print("Enter receiver UPI ID: ");
            String receiverUPI = sc.nextLine();

            System.out.print("Enter payment amount: ");
            double paymentAmount = sc.nextDouble();

            payment.pay(receiverUPI, paymentAmount);
```

```
        } catch (InvalidUPIException
                | InvalidAmountException
                | InsufficientBalanceException e) {

            System.out.println("Transaction failed: "
                    + e.getMessage());

        } finally {
            System.out.println("\n----- TRANSACTION COMPLETED -----");
            wallet.displayWalletDetails();
            payment.checkBalance();
            sc.close();
        }
    }
}
```

8. Terminal-Style Output Screen

```
===== DIGITAL WALLET SYSTEM =====
Enter your name: Prasanna
Enter mobile number: 9876543210
Enter your UPI ID: prasanna@upi
Enter amount to add: 10000
Money added successfully: Rs. 10000.0
Available Balance: Rs. 10000.0
Enter receiver UPI ID: ravi@upi
Enter payment amount: 2500
Payment successful!
Receiver UPI: ravi@upi
Amount paid: Rs. 2500.0

----- TRANSACTION COMPLETED -----

----- WALLET DETAILS -----
User Name: Prasanna
Mobile Number: 9876543210
UPI ID: prasanna@upi
Balance: Rs. 7500.0
Available Balance: Rs. 7500.0
```

9. Result

Thus, the Digital Wallet and UPI Payment System was successfully designed and implemented in Java. The program demonstrates interfaces, encapsulation, constructors, object creation, String validation, custom exceptions, try-catch-finally handling, money addition, UPI payments, balance checking, and transaction validation.

10. Important Note

This is an educational simulation. It does not connect to an actual bank account or real UPI network. The UPI ID format is checked only for basic validity.